Design and implementation of five stage pipelined RISC-V processor using Verilog

NAME: SAKTHIVEL B

REG N.O: 21BEC2165

EMAIL: sakthivelbhaaskaran@gmail.com

PHONE: 9360948661

OBJECTIVE:

Our primary objective is to design a 32- bit RISC V Processor using Verilog for hardware description and to use Universal Verification Methodology (UVM) for comprehensive verification.

The key goals are:

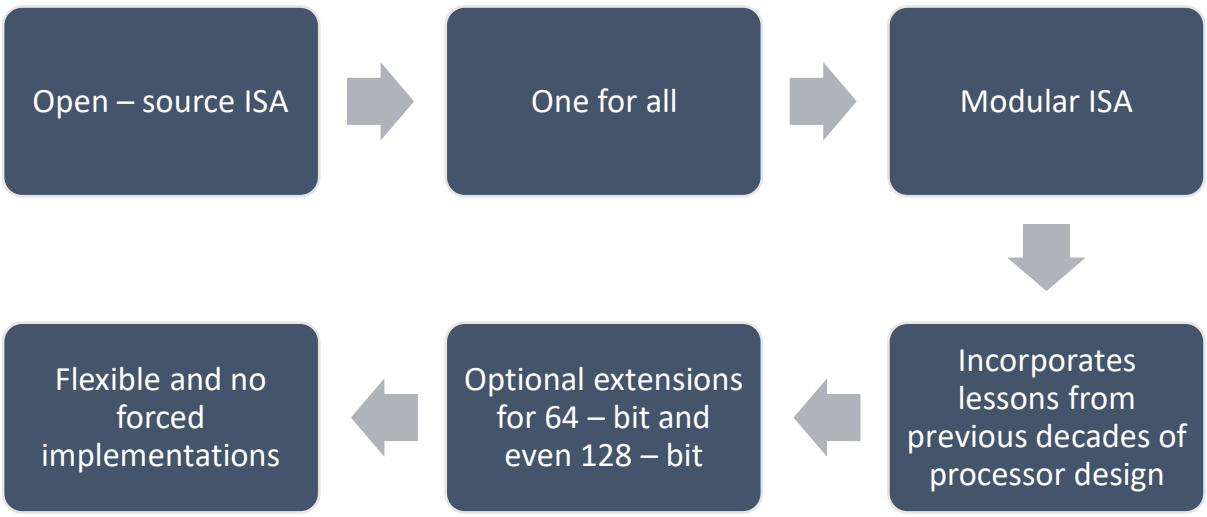
1→ **Accurate Processor Design**

2→ **Efficiency and Performance**

CURRENT STATUS:

- We have implemented a five staged 32-bit RISC V, which includes six instruction set types : **R-type ,I-type, B-type, J-type , L-type and S-type**. This design features a five-stage pipeline consisting of **instruction fetch, instruction decode, instruction execute, memory access, write back** stages.

MOTIVATION:



IDENTIFICATION OF GAPS IN RISC V:

- **Software Ecosystem**: Less mature compared to ARM and x86, with fewer tools, libraries, and applications.
- **Hardware Maturity**: RISC-V hardware implementations are still developing in terms of performance and efficiency.
- **ISA Compliance**: Ensuring that RISC-V cores strictly follow the ISA specifications is challenging, requiring thorough verification of instructions and registers.
- **Tool Gaps:** Limited availability and maturity of simulators, verification methods, and development tools.

SOFTWARE USED:

- The primary development was done using **Modelsim ,** and **Vivado xilinx**

was used to synthesis the output

Modelsim:

- Primary used for simulating **HDL designs** (VHDL , Verilog). It helps developers

Verify and debug their hardware logic by providing detailed waveform outputs and

Simulation logs.

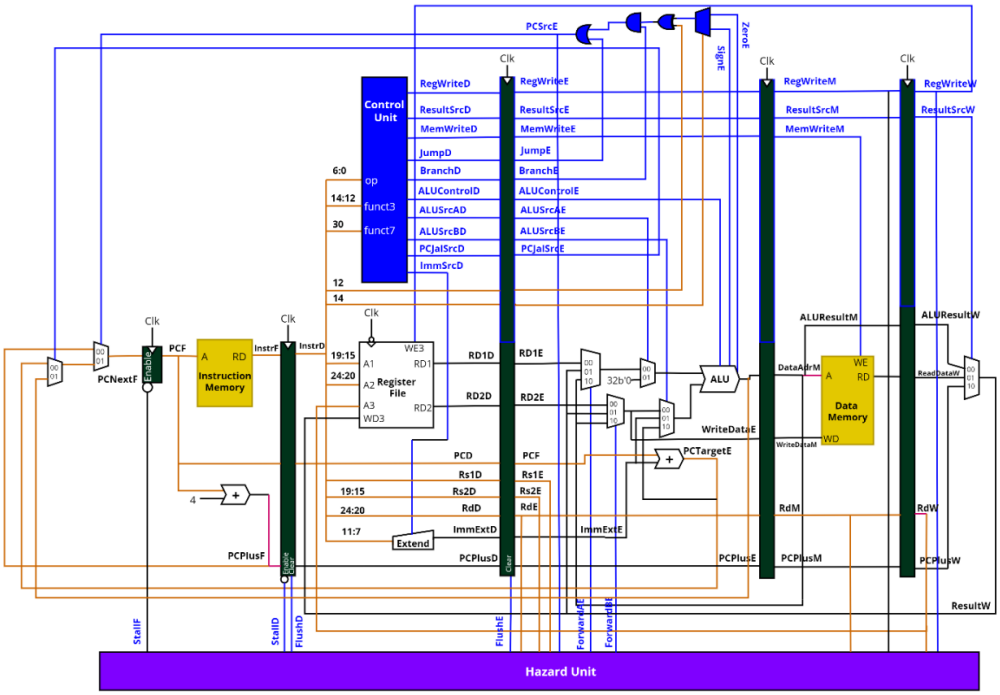
Vivado xilinx:

- This is used for synthesis and analysis of hardware description language designs

Programming language used: Verilog

METHODOLOGY

Architecture of RV32I processor:



ISA (Instruction Set Architecture):

Field Name	Bit Range	Description	Width (bits)
Opcode	[6:0]	Determine the type of instruction (eg:- arithmetic , load , store , branch..)	7
Rd	[11:7]	Destination register	5
Funct3	[14:12]	Determine the operation variant within the instruction type (eg:- addition , subtract..)	3
Rs1	[19:15]	Source register 1	5
Rs2	[24:20]	Source register 2	5
funct7	[31:25]	Additional operation information such as whether the operation is signed or unsigned	7

Instr[31:25]	Instr[24:20]	Instr[19:15]	Instr[14:12]	Instr[11:7]	Instr[6:0]	Type
funct7	rs2	rs1	funct3	rd	opcode	R-Type
Imm[11:0]		rs1	funct3	rd	opcode	I-Type
Imm[11:5]	rs2	rs1	funct3	Imm[4:0]	opcode	S-Type
Imm[12] Imm[10:5]	rs2	rs1	funct3	Imm[4:1] Imm[11]	opcode	B-Type
imm[31:12]				rd	opcode	U-Type
Imm[20] Imm[10:1]	Imm[11]	Imm[19:12]		rd	opcode	J-Type

R-Type instruction

Operation	Implementation
ADD	RegFile[rd] = RegFile[rs2]+RegFile[rs1];
SUB	RegFile[rd] = RegFile[rs1]-RegFile[rs2];
SLL	RegFile[rd] = RegFile[rs1] << (RegFile[rs2] & 0x1F);
SLT	RegFile[rd] = ((signed long)RegFile[rs1] < (signed long)RegFile[rs2]) ? 1 : 0;
SLTU	RegFile[rd] = (RegFile[rs1]<RegFile[rs2]) ? 1 : 0;
XOR	RegFile[rd] = RegFile[rs1] ^ RegFile[rs2];
SRL	RegFile[rd] = RegFile[rs1] ^ RegFile[rs2];
SRA	RegFile[rd] = RegFile[rs1] >> (RegFile[rs2] & 0x1F);
OR	RegFile[rd] = RegFile[rs1] | RegFile[rs2];
AND	RegFile[rd] = RegFile[rs1] & RegFile[rs2];

I-TYPE Instruction:

Operation	Implementation
ADDI	RegFile[rd] = Immediate_Value+RegFile[rs1];
SLLI	RegFile[rd] = RegFile[rs1] << (Immediate_Value);
SLTI	RegFile[rd] = ((signed long)RegFile[rs1] < (signedlong)Immediate_Value) ? 1 : 0;
SLTIU	RegFile[rd] = (RegFile[rs1]<Immediate_Value) ? 1 : 0;
XORI	RegFile[rd] = RegFile[rs1] ^ Immediate_Value;
SRLI	RegFile[rd] = RegFile[rs1] >> (Immediate_Value);
SRAI	RegFile[rd] = RegFile[rs1] >>>(Immediate_Value);
ORI	RegFile[rd] = RegFile[rs1] | Immediate_value;
ANDI	RegFile[rd] = RegFile[rs1] & Immediate_Value;

S,J, and L Type instruction:

Operation	Implementation	Instruction Type
SW	Data_Memory[(Immediate_Value + RegFile[rs1])] = RegFile[rs2] ;	S Type
JAL	RegFile[rd] = PC + 0x4; PC = Immediate_Value + PC ;	J Type
LW	RegFile[rd] = Data_Memory[Immediate_Value + RegFile[rs1]] ;	L Type

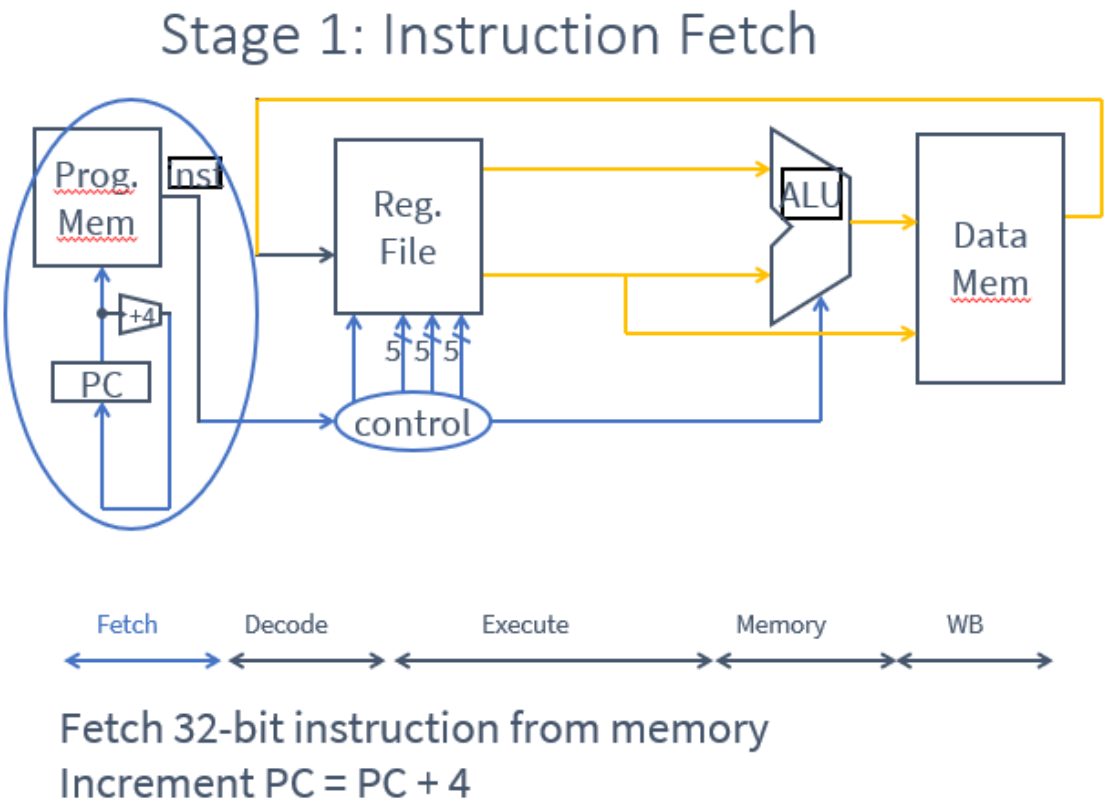
B-Type instruction:

Operation	Implementation
BEQ	if(RegFile[rs1] == RegFile[rs2]) { PC = Immediate_Value + PC ; }
BNE	if(RegFile[rs1] != RegFile[rs2]) { PC = Immediate_Value + PC ; }
BLTU	if(RegFile[rs1] < RegFile[rs2]) { PC = Immediate_Value + PC ;}
BGTU	if(RegFile[rs1] >= RegFile[rs2]) { PC = Immediate_Value + PC ; }

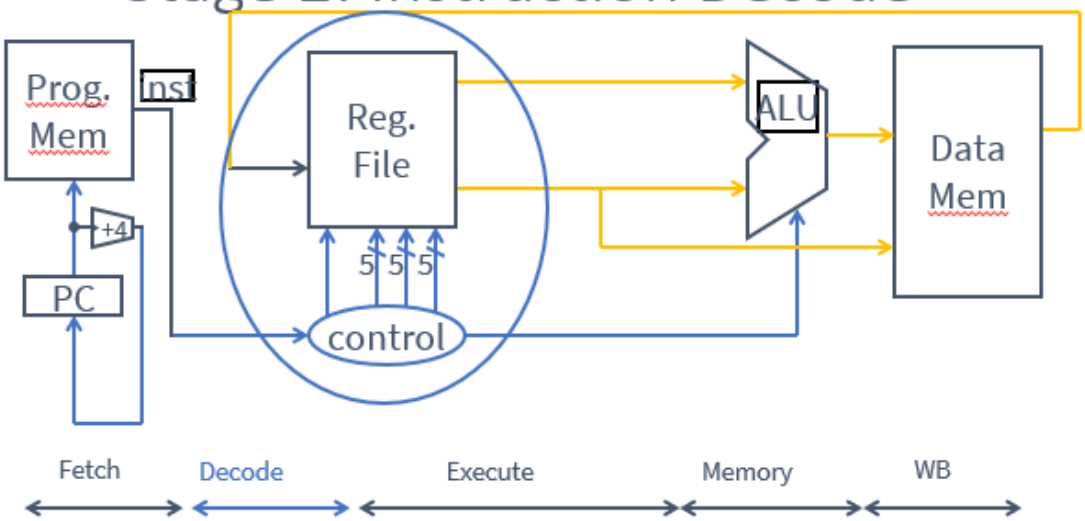
STAGES OF RISC V:

There are 5 stages in RISC V processor they are listed below:

1. Instruction Fetch
2. Instruction Decode
3. Execution (ALU)
4. Memory Access
5. Register Writeback

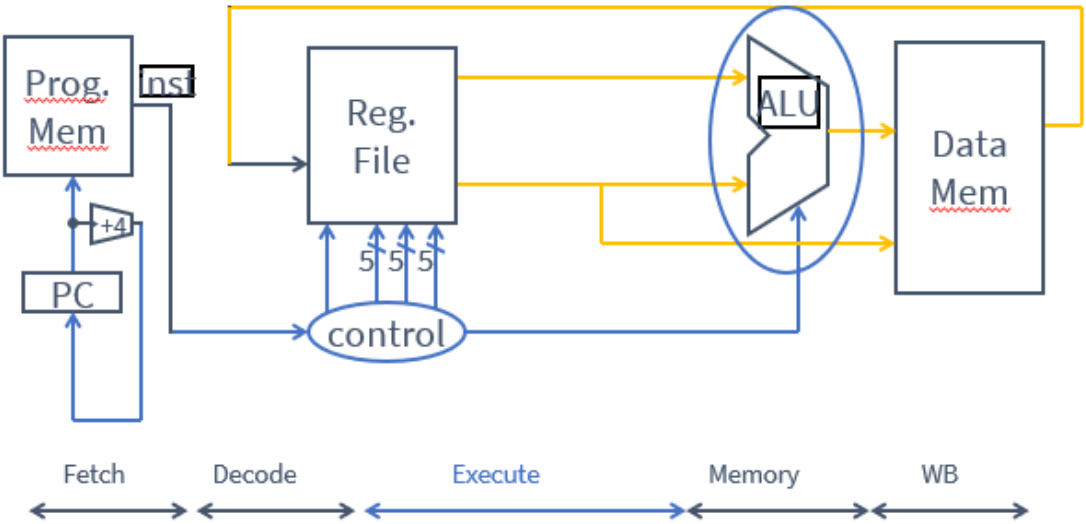


Stage 2: Instruction Decode



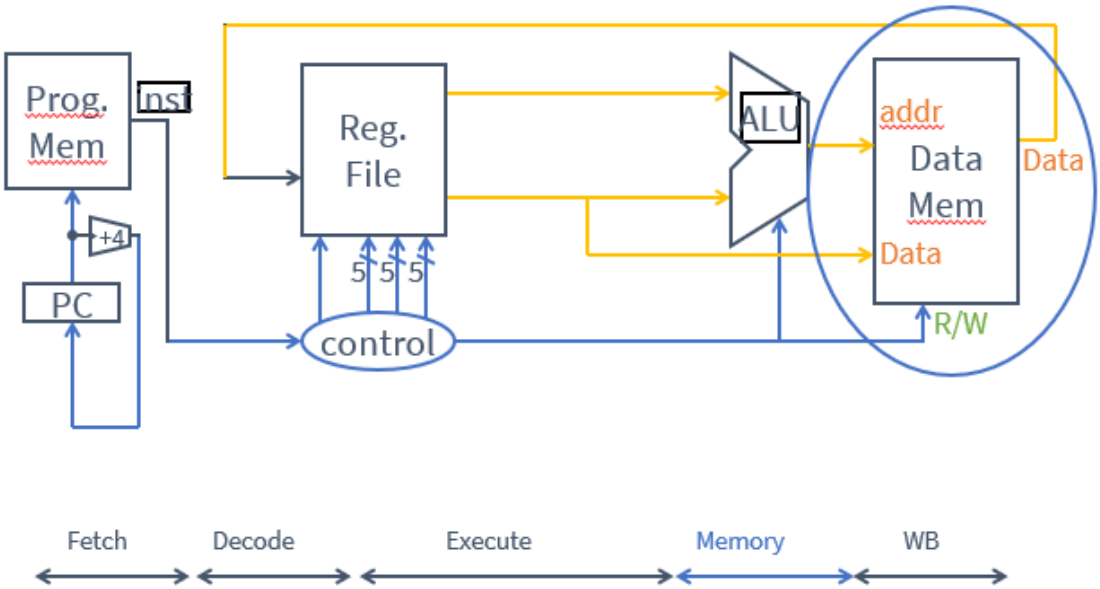
Gather data from the instruction
Read opcode; determine instruction type, field lengths
Read in data from register file
(0, 1, or 2 reads for `jump`, `addi`, or `add`, respectively)

Stage 3: Execution (ALU)



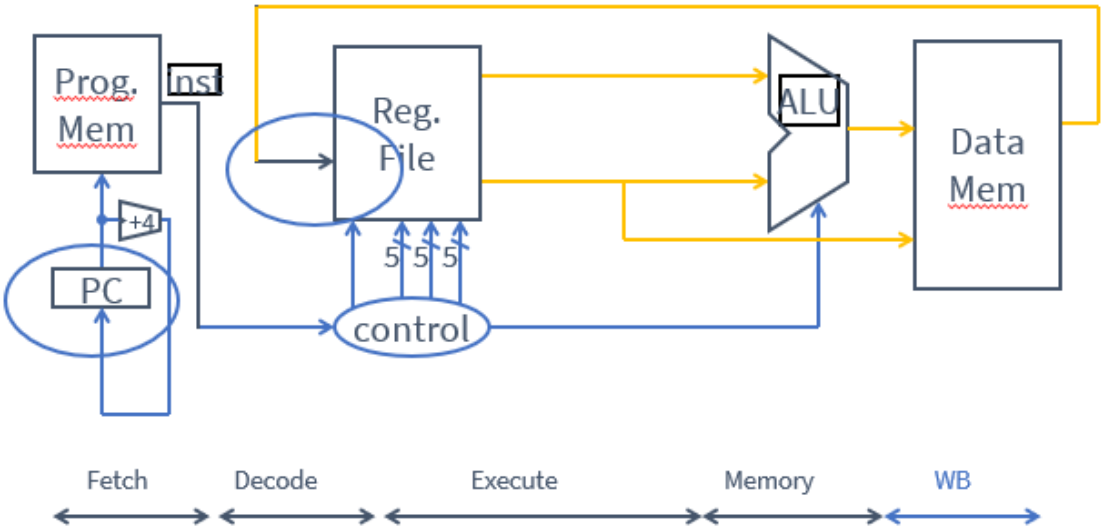
Useful work done here (+, -, *, /), shift, logic operation, comparison (slt)

Stage 4: Memory Access



Used by load and store instructions only
Other instructions will skip this stage

Stage 5: Writeback



Write to register file

- For arithmetic ops, logic, shift, etc, load.

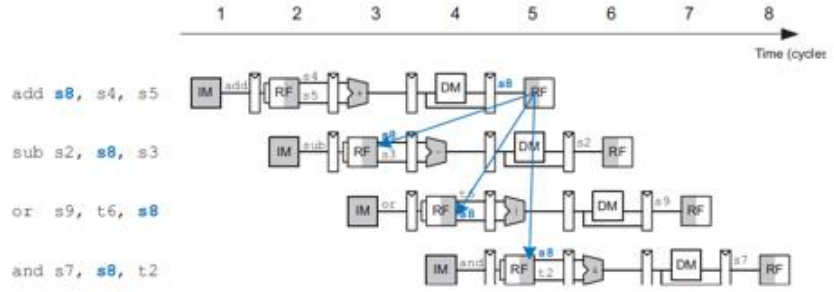
Update PC

- For branches, jumps

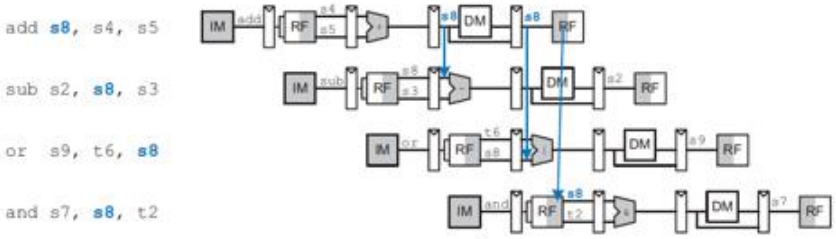
Hazard Handli

Occurs when an instruction depends on the results of the previous instruction which is not yet completed.

1. Data Hazards
    1. Forwarding
    2. Stalling
2. Control Hazards
    1. Flushing.



Data Hazard

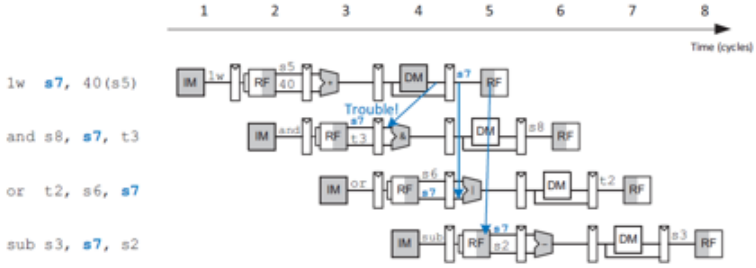


Solution

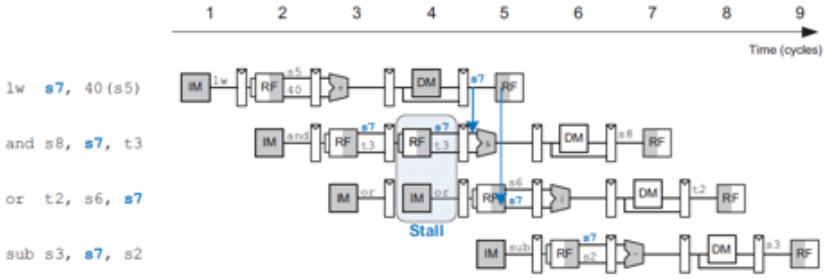
Stalling for load word hazard

Since the result of load instr. not available till the end of Memory Access Stage, we stall the next instructions by cycle.

When result appears in Write Back Stage, we forward it to the execution stage.



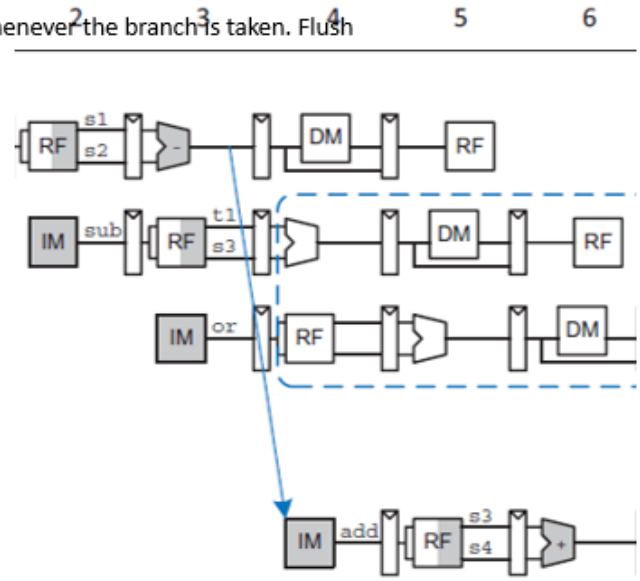
Load word hazard



Solution

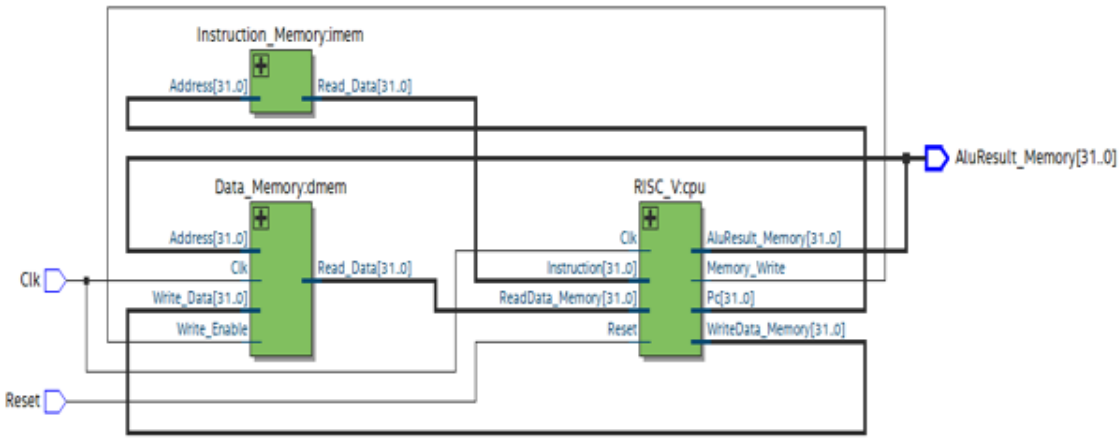
Control Hazards

We flush the following two instructions whenever the branch is taken. Flush decode and execute stage.

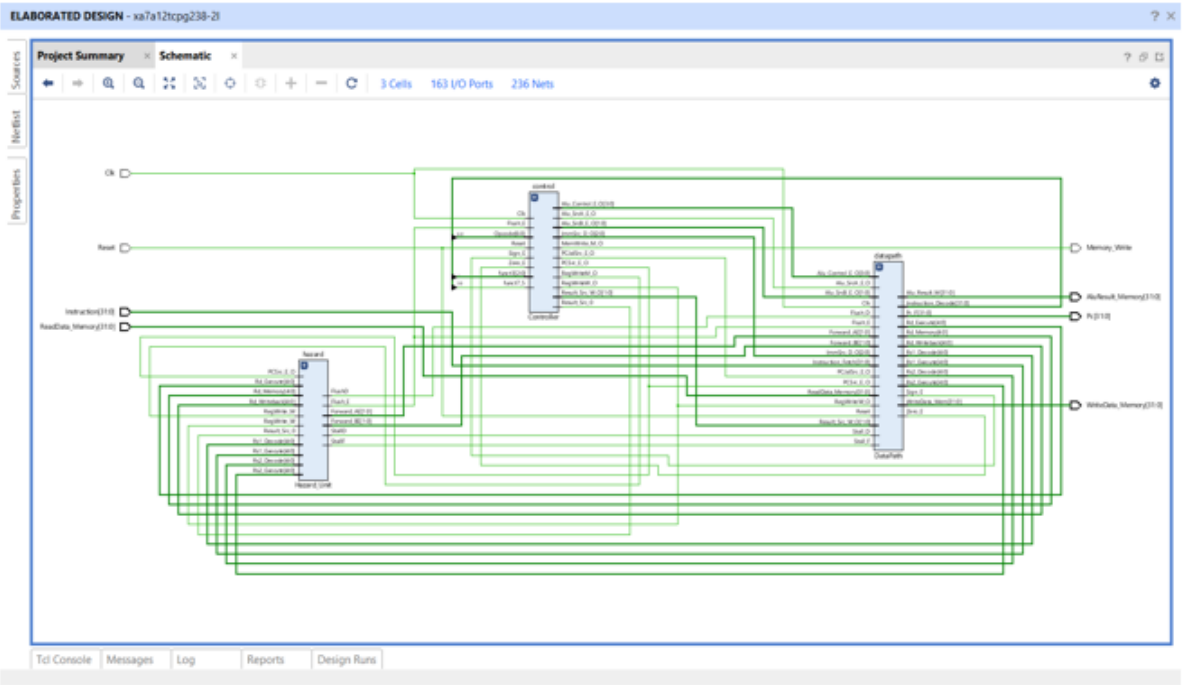


RESULT:

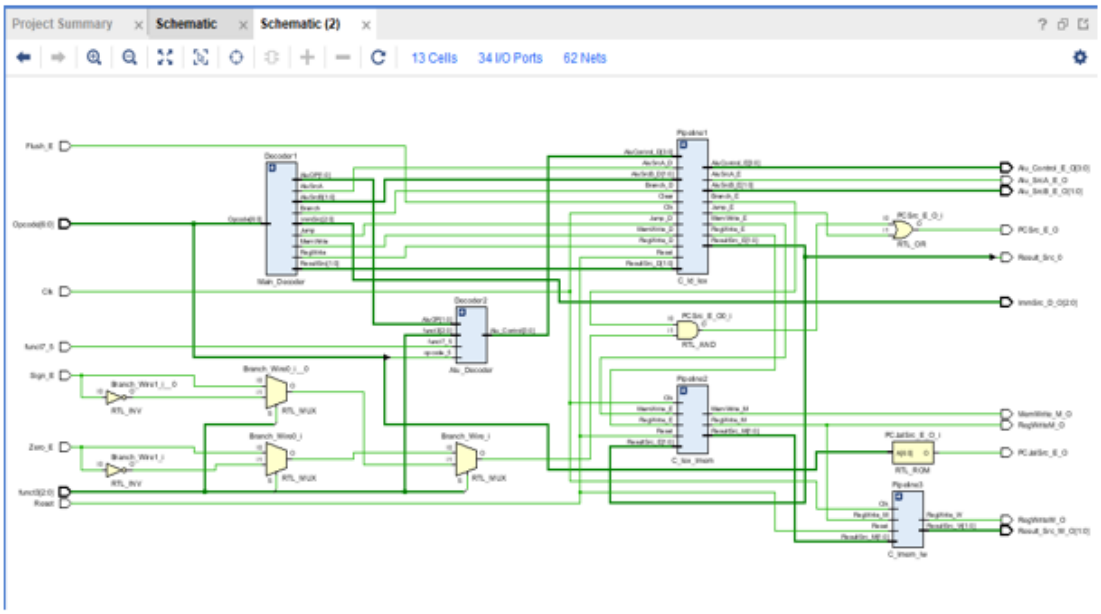
RTL SYNTHESIS OF TOP MODULE:



RTL SYNTHESIS OF RISC BLOCK



RTL SYNTHESIS OF CONTROLLER BLOCK



SCOPE FOR IMPROVEMENT

The current RISC-V processor design can be enhanced to support more advanced operations.

1)Floating-Point Arithmetic:

Add a Floating-Point Unit (FPU) to improve precision and efficiency for real number computations, supporting operations like addition, subtraction, multiplication, and division.

2)MAC Unit (Multiply-Accumulate)

The MAC unit performs both multiplication and addition in a single instruction.It multiplies two numbers and adds the result to an accumulator, making it ideal for tasks involving repetitive calculations like digital signal processing (DSP) and machine learning algorithms. Efficient handling of complex arithmetic operations

REFERENCES:

1. D. Bhandarkar and D.W. Clark, “Performance from Architecture: Comparing a RISC and a CISC with Similar Hardware

Organization,”Proceedings of the 4th Int’l. Conference on ASPLOS, Santa Clara, California, April 8-11, 1991.

2. Kulshreshtha, A., Moudgil, A., Chaurasia, A. and Bhushan, B., 2021, March. Analysis of 16-Bit and 32-Bit RISC Processors. In 2021 7th International Conference on Advanced Computing and Communication Systems (ICACCS) (Vol. 1, pp. 1318-1324). IEEE.

3. Khairullah, S.S., 2022, June. Realization of a 16-bit MIPS RISC pipeline processor. In 2022 International Congress on Human-Computer.

4. M. N. Topiwala and N. Saraswathi, "Implementation of a 32-bit MIPS based RISC processor using Cadence," 2014 IEEE International Conference on Advanced Communications, Control and Computing Technologies, 2014.

5. Islam, S., Chattopadhyay, D., Das, M.K., Neelima, V. and Sarkar, R., 2006, September.Design of High-Speed-Pipelined Execution Unit of 32-bit RISC Processor. In 2006 Annual IEEE India Conference (pp. 1-5). IEEE.

6. S. P. 6. Ritpurkar, M. N. Thakare and G. D. Korde, "Design and simulation of 32-Bit RISC architecture based on MIPS using VHDL," 2015 International Conference on Advanced Computing and Communication Systems, 2015.Interaction, Optimization and Robotic Applications (HORA) (pp. 1-6). IEEE.

7. Al-sudany, S.M., Al-Araji, A.S. and Saeed, B.M., 2021. FPGA-Based Multi-Core MIPS Processor Design. IRAQI JOURNAL OF

COMPUTERS, COMMUNICATION, CONTROL & SYSTEMS ENGINEERING, 21(2).

8. Wang, W., Han, J., Cheng, X. and Zeng, X., 2021. An energy-efficient cryptoextension design for RISC-V. Microelectronics Journal, 115, p.105165